SHRINAV WORKSHOP

Practice Assignment

API Fundamentals & Postman Mastery

Complete every task in Postman using the JSONPlaceholder API. Read each instruction carefully — later tasks build on what you did in earlier ones.

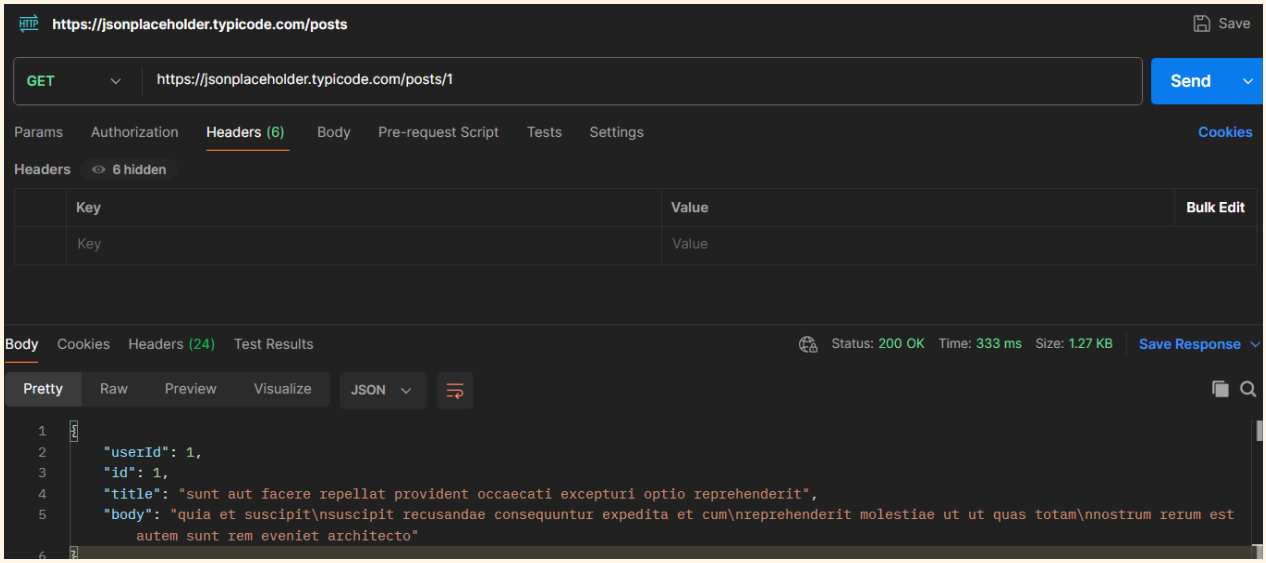
Before you start: *you only need a browser and Postman. Base URL for every task: https://jsonplaceholder.typicode.com — no signup or API key required.*

Tasks

TASK 1 Reading Data Without Headers

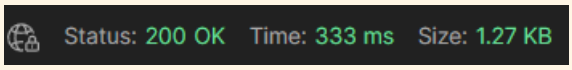
a) Send a GET request to /posts/1 — leave the Headers and Body tabs completely empty.

Ans:



b) Record the status code you get back.

Ans:



c) In one sentence, explain why this request works even with nothing in the Headers tab.

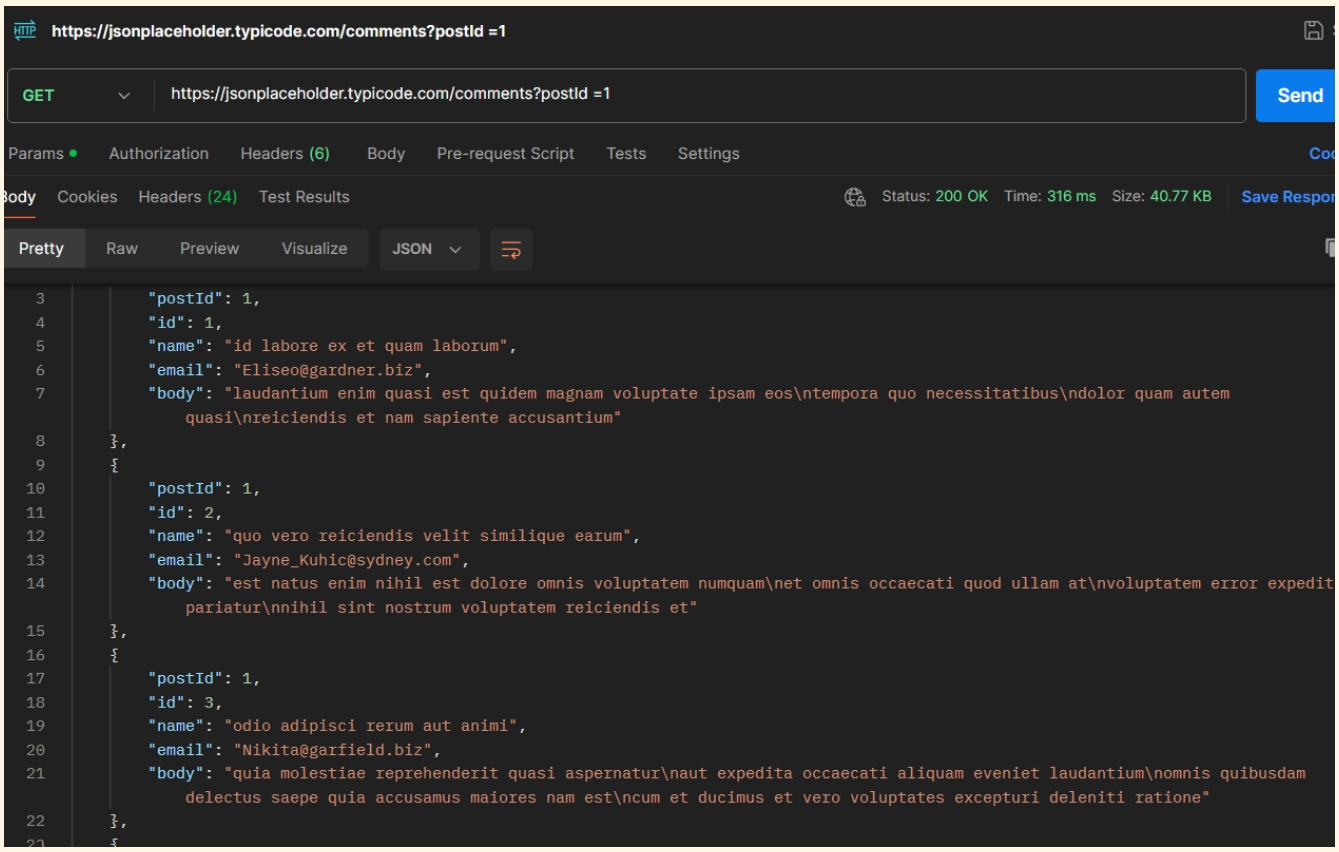
Ans:

This request works even with nothing in the Headers tab because GET requests only need the URL to identify the resource, and the server doesn't require extra headers or a body to return a public resource.

TASK 2 Filtering With Query Parameters

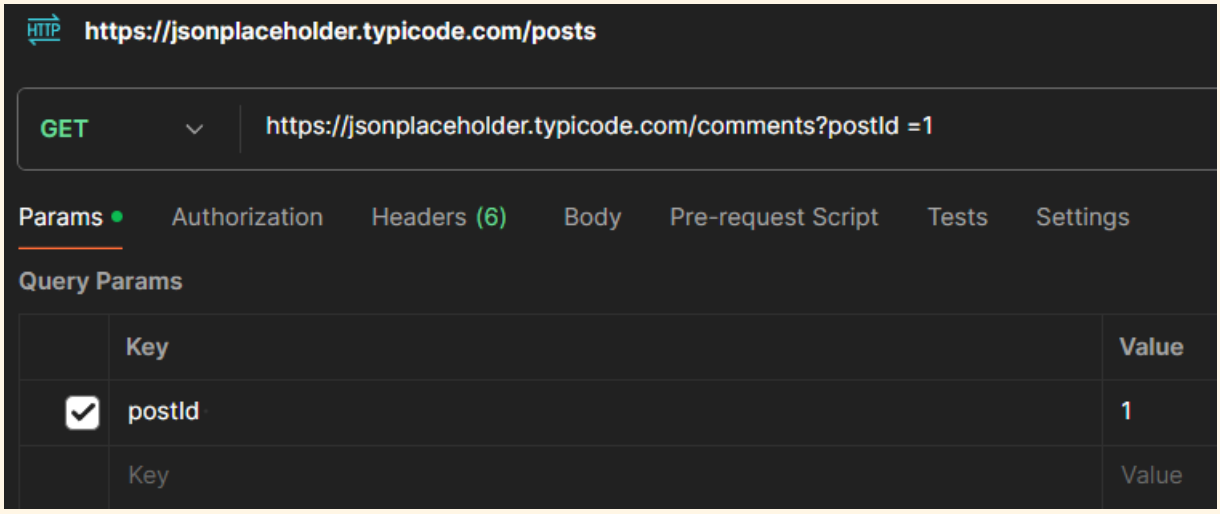
a) Send a GET request to /comments, and add a query parameter in the Params tab: postId = 1.

Ans:



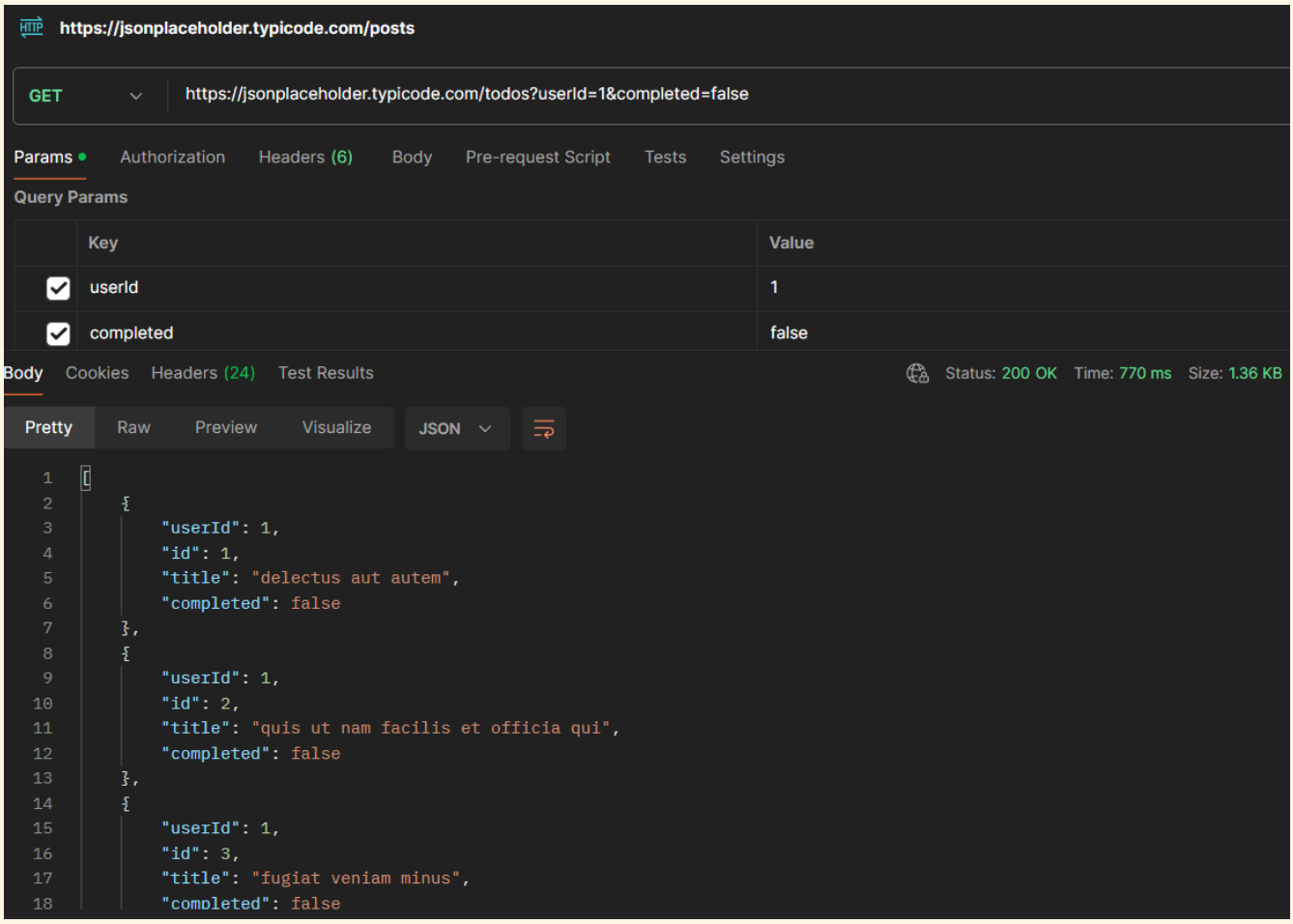
b) Confirm the URL automatically updates to include ?postId=1.

Ans: Yes.



c) Now combine two filters at once on /todos: userId = 1 and completed = false.

Ans:



d) In one sentence, explain what changed in the results compared to not using any params.

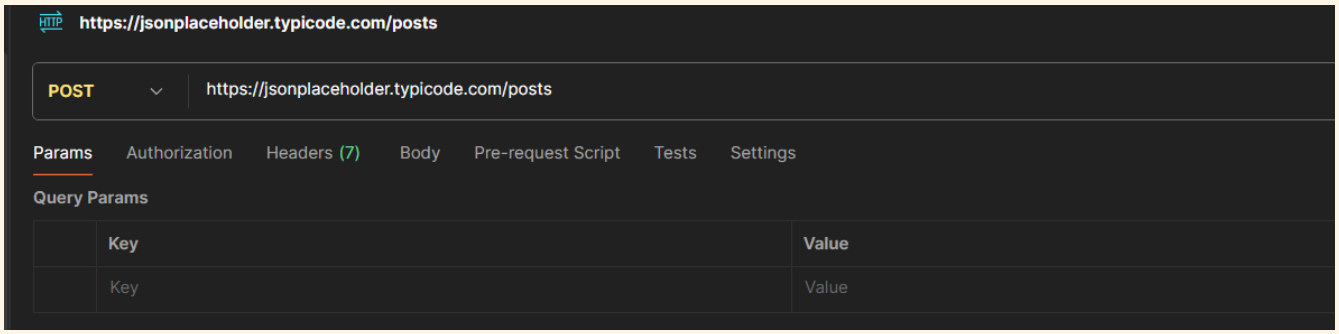
Ans:

Using query parameters filters the results returned by the API. Instead of receiving all records, the server returns only the data that matches the specified conditions, making the response smaller and more relevant.

TASK 3 Sending Data With Headers & a Body

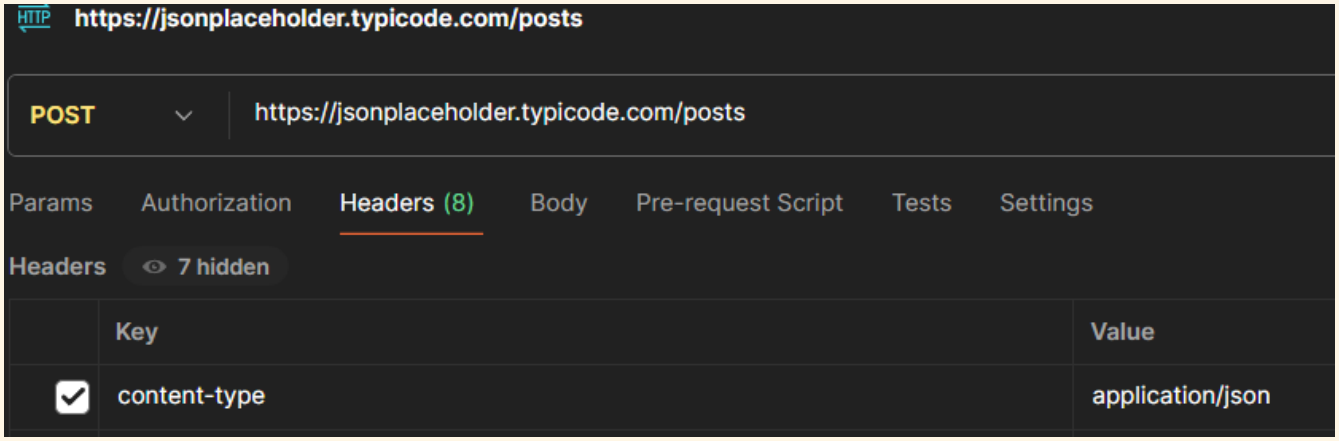
a) Send a POST request to /posts.

Ans:



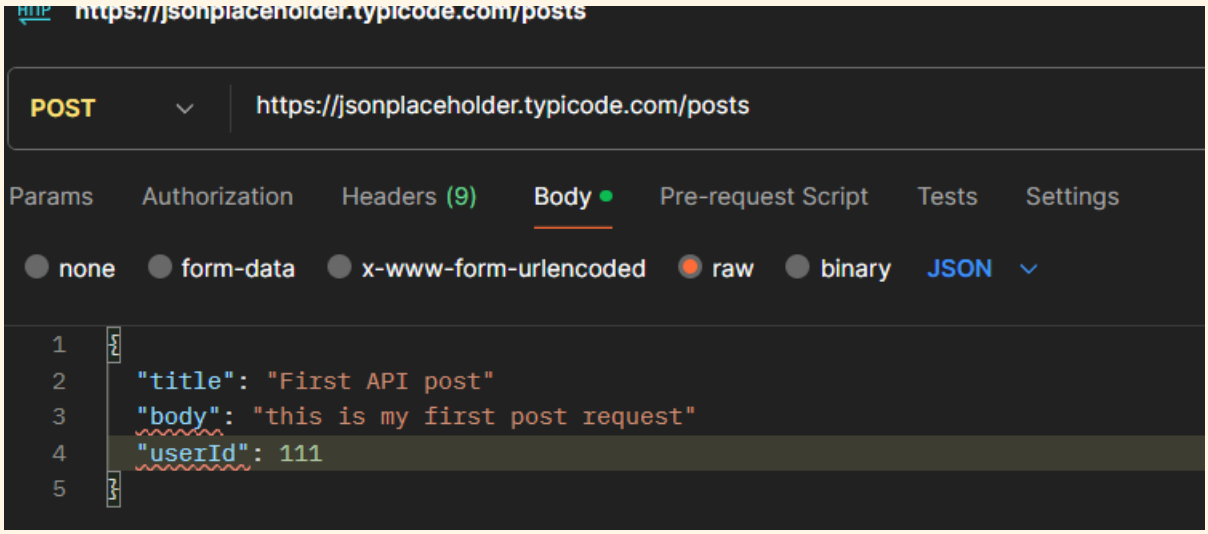
b) In the Headers tab, add: Content-Type: application/json.

Ans:



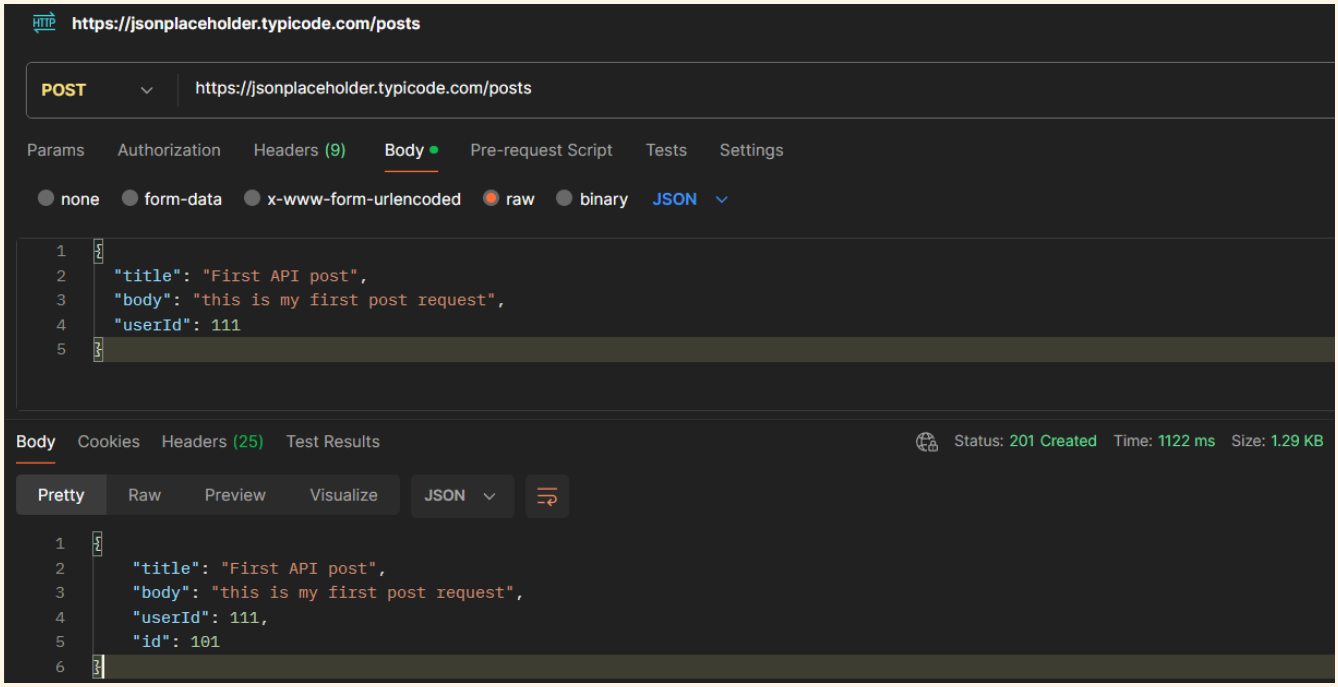
c) In the Body tab (raw, JSON), send a new post with a title, body, and userId of your choice.

Ans:



d) Record the status code and the new id the server assigned.

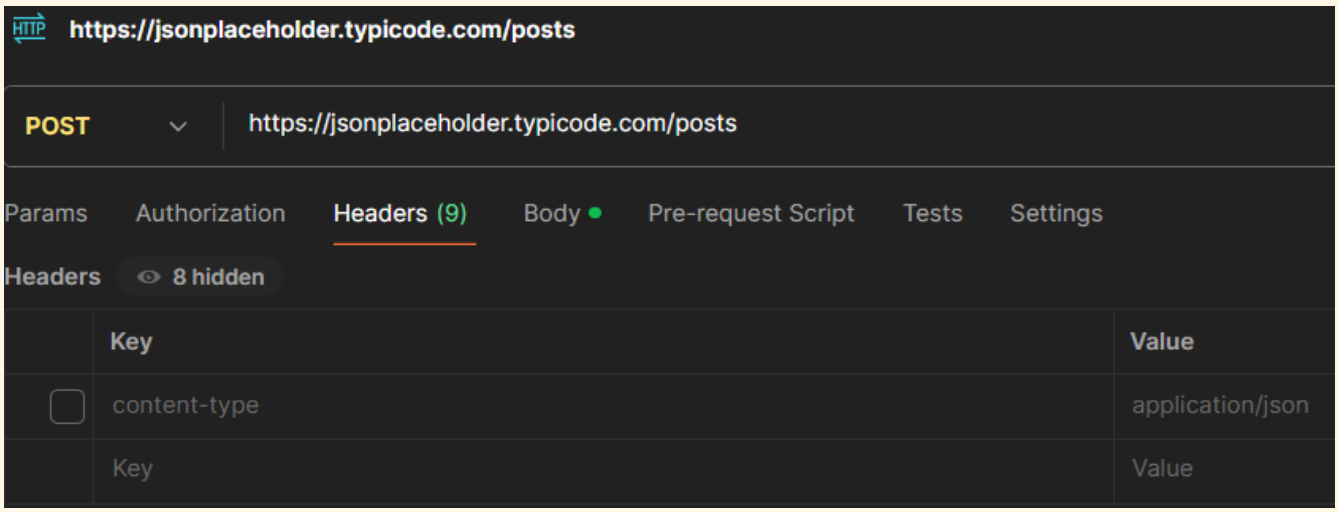
Ans: **Status Code:** 201 Created ; **ID Assigned:** 101



TASK 4 Headers vs. No Headers

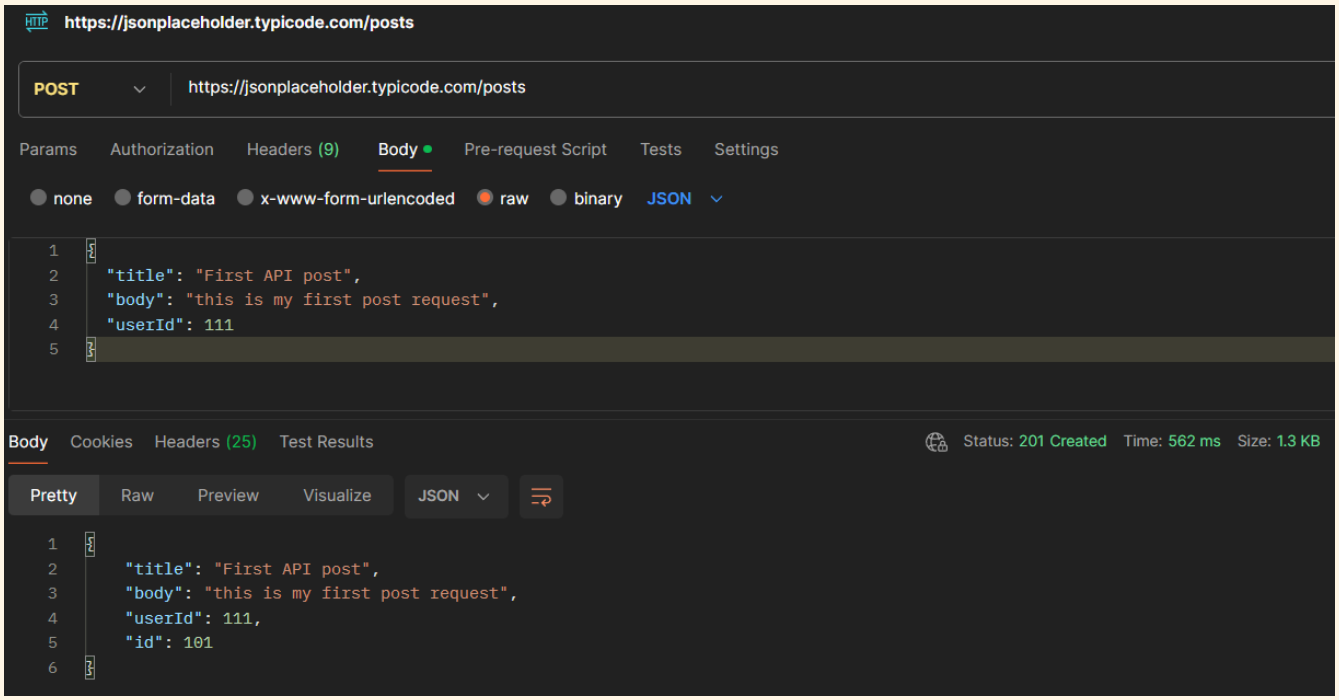
a) Repeat Task 3's POST request, but this time remove the Content-Type header before sending.

Ans:



b) Compare the response to Task 3 — is it different? Record what you observe.

Ans:



With the Content-Type header, the server correctly identifies the request body as JSON and processes it, returning a 201 Created response.

Without the Content-Type header, the request is still processed and returns 201 Created, because JSONPlaceholder accepts the request even without the header.

c) In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

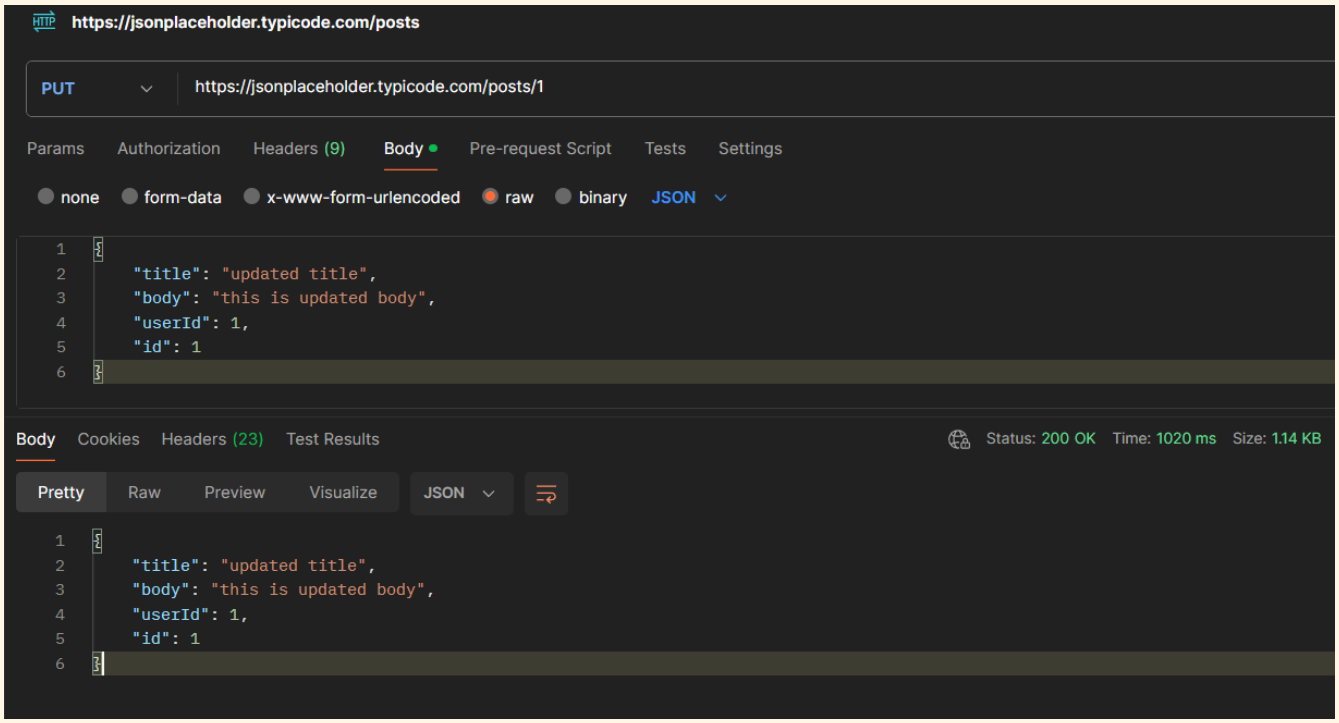
Ans:

Headers are important because they tell the server how to interpret the request data. The Content-Type: application/json header informs the server that the request body contains JSON. Although JSONPlaceholder accepts the request without this header, a real API may reject the request or return an error such as 400 Bad Request or 415 Unsupported Media Type if the correct header is missing.

TASK 5 PUT vs. PATCH

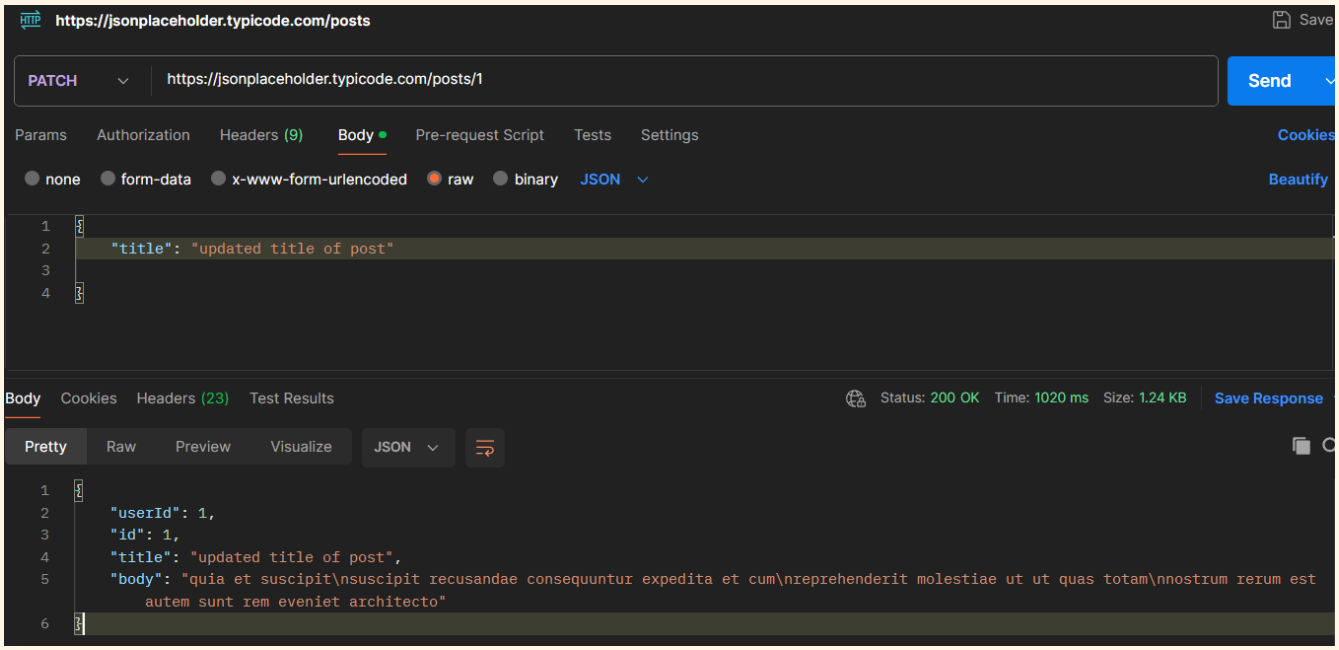
a) Send a PUT request to /posts/1 that replaces the entire post (include id, title, body, and userId in the body).

Ans:



b) Send a PATCH request to the same /posts/1 that changes only the title.

Ans:



c) In one sentence, explain the difference between what PUT sent and what PATCH sent.

Ans:

PUT replaces the entire resource, so the request body must contain all fields of the resource. PATCH updates only the specified fields, so the request body can contain only the fields that need to be changed.

Final Challenge — A Realistic Request Flow

Concepts used: *this task combines query params, headers (with and without), and the request body — everything from Tasks 1 through 4 — in one flow.*

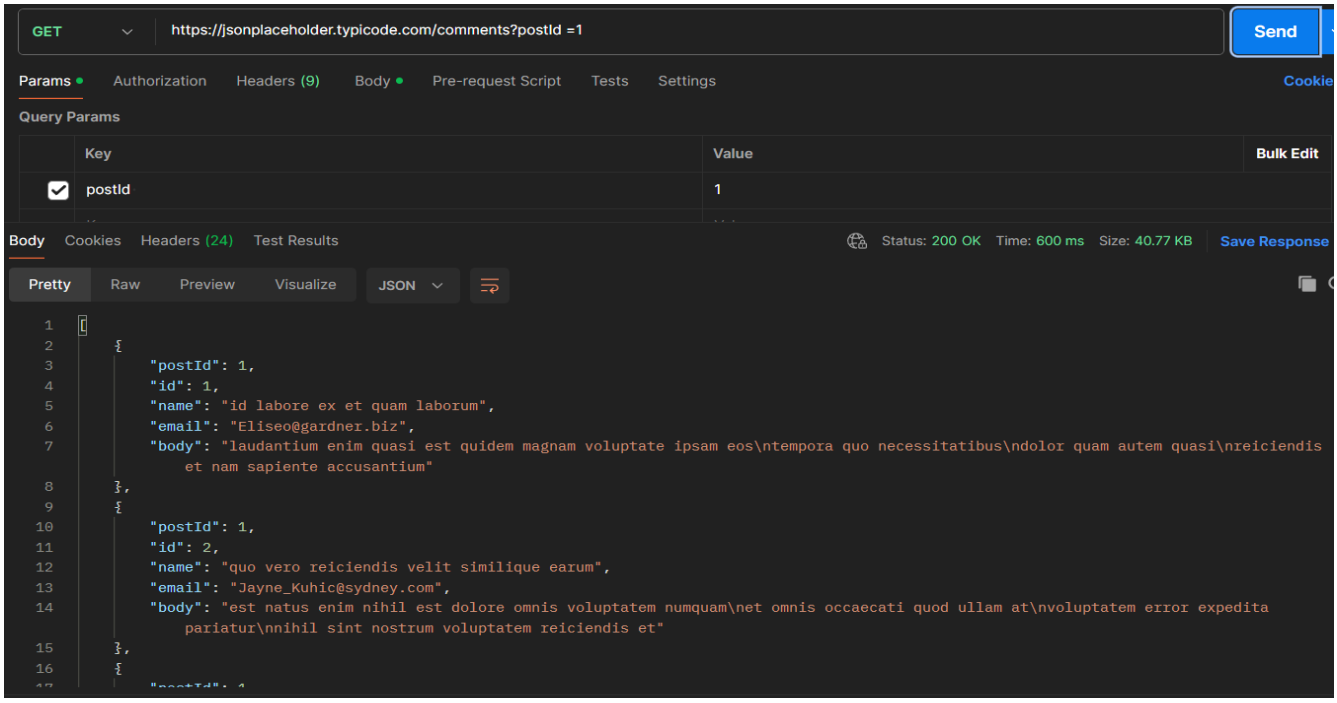
TASK 6 The Mini Feedback System

Scenario: you're testing the backend for a simple feedback form, where users read existing comments on a post and can leave their own.

Complete the following steps, in order:

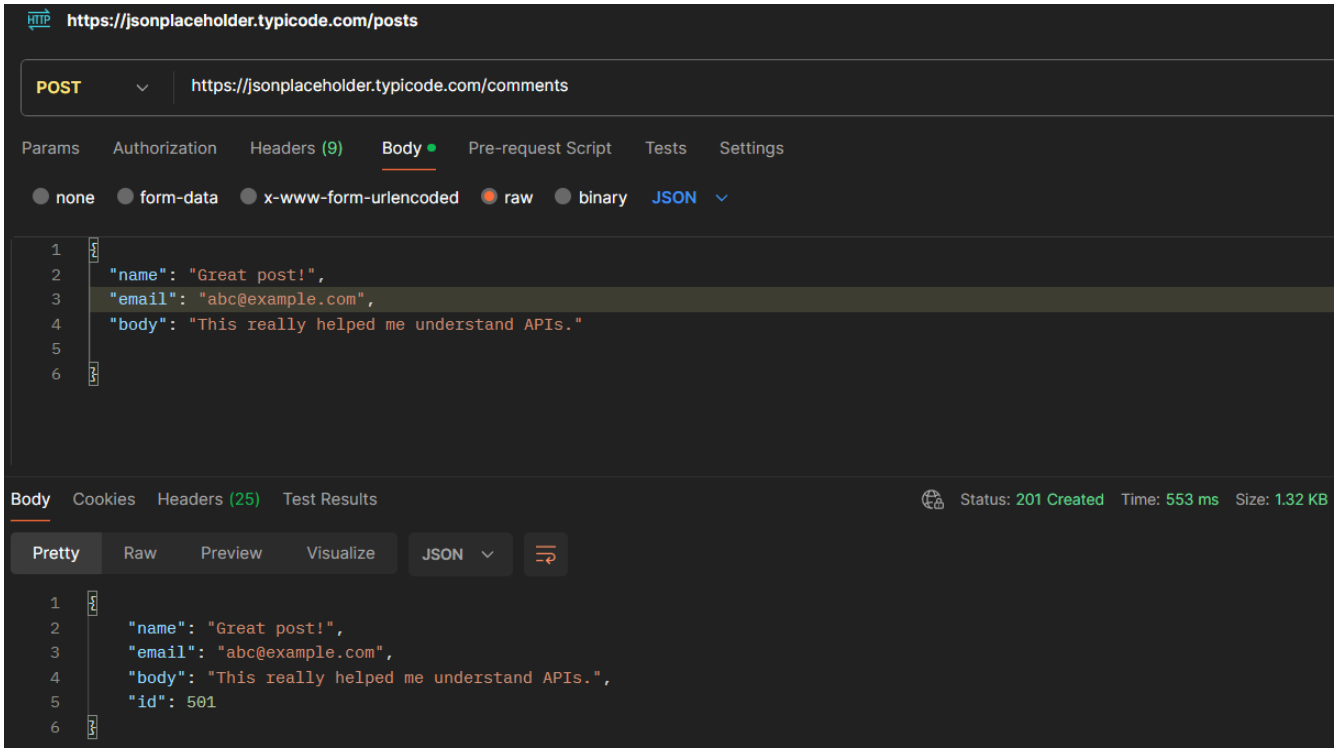
a) Send a GET request to /comments, using a query parameter to fetch only the comments for postId = 1.

Ans:



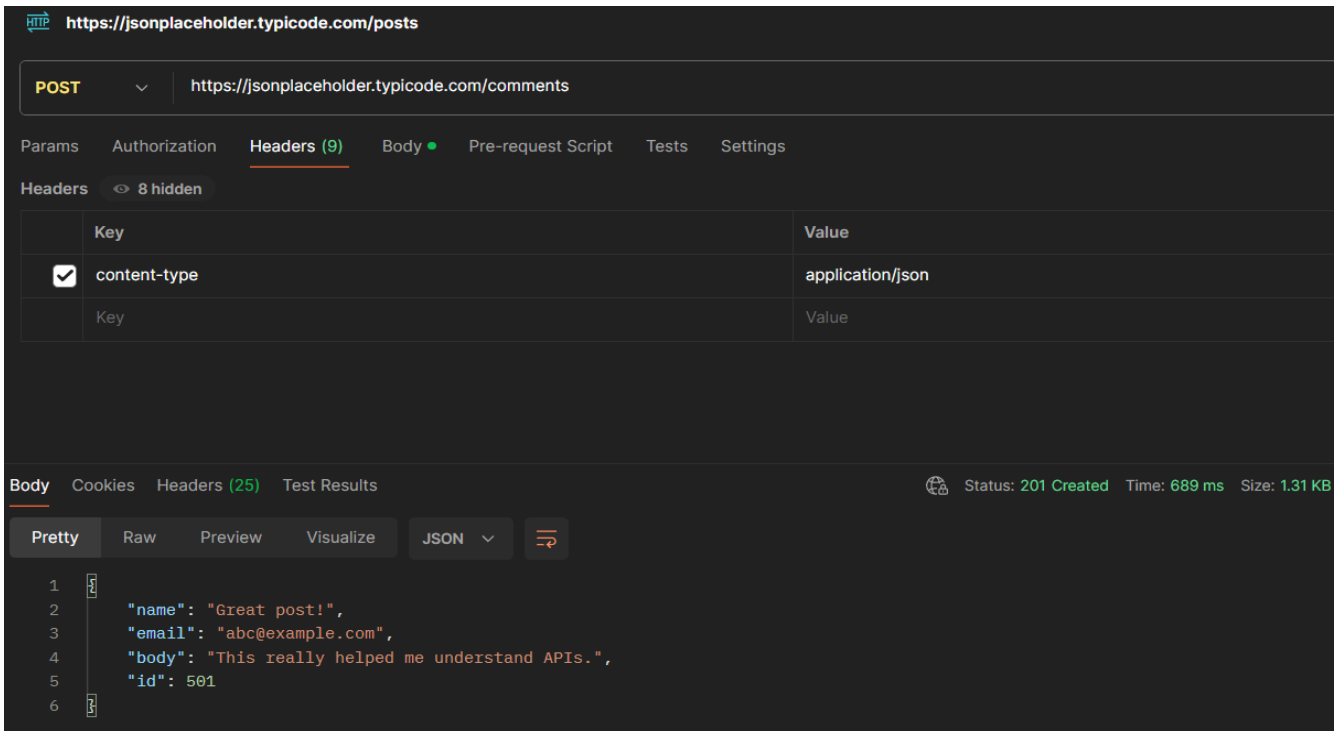
b) Send a POST request to /comments to add a new comment — with no headers at all. Record the status code.

Ans:



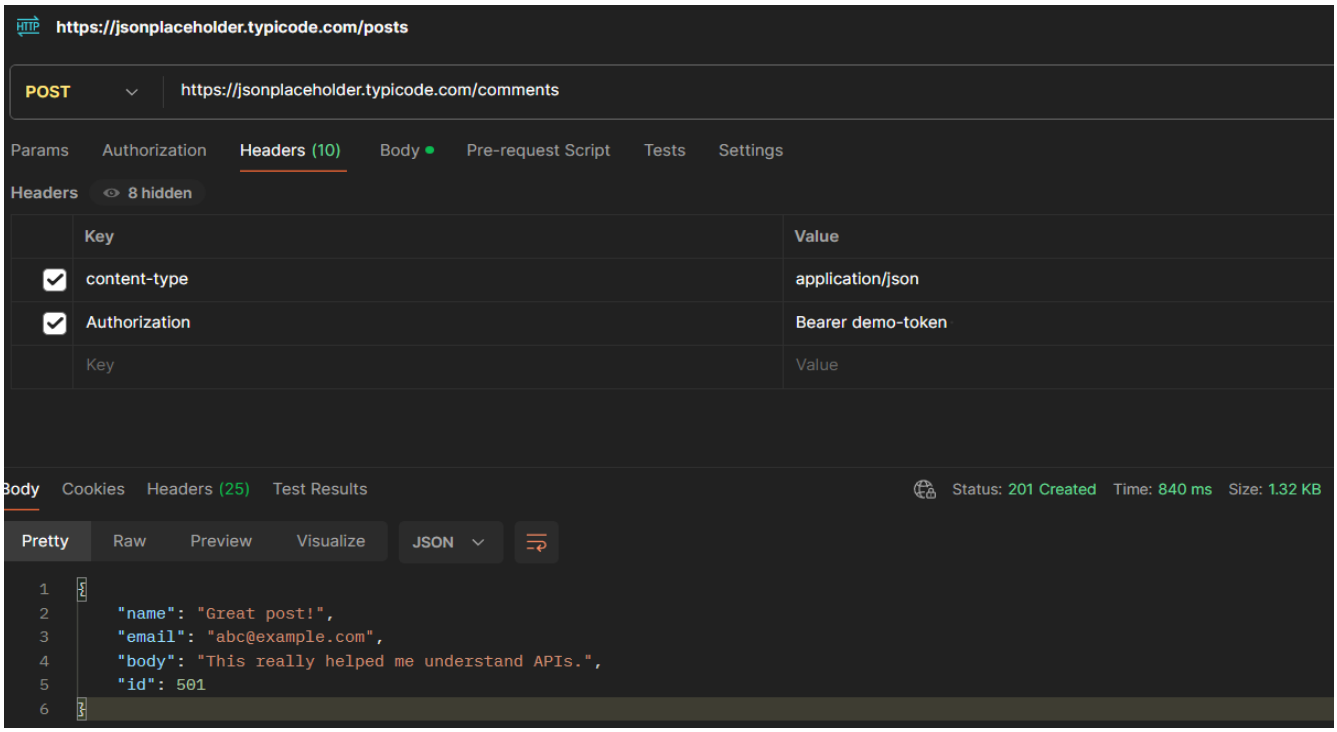
c) Resend the same POST from step (b), this time with the Content-Type header included. Compare the two responses.

Ans:



d) Add one more header to this request: Authorization: Bearer demo-token — to simulate a login-protected submission, even though this practice API won't actually check it.

Ans:



e) Record the status code for every step above (a–d) in a short table or list.

Ans:

Step	Status Code
(a)	200 OK
(b)	201 Created
(c)	201 Created
(d)	201 Created

f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

Ans:

If this were a real API, the Authorization header would contain a valid authentication token. The server would verify the token before processing the request. If the token was missing or invalid, the server would reject the request with an error such as 401 Unauthorized or 403 Forbidden.